

Điểm nổi bật

EMESoft Agent Suite là ba ứng dụng dùng chung một danh tính và một kho cấu hình:

EmeHub giữ người dùng, credential và project; **Q-Agent** làm việc của QA; **D-Agent** làm việc của dev.

Tài liệu này nói về những chỗ chúng tôi làm khác. Mỗi khẳng định dưới đây đều chỉ được tới file trong repo. Những gì chưa làm nằm ở KNOWN-GAPS.md và không được nhắc ở đây như thể đã xong.

1. Một tài khoản Claude không còn là nút thắt cổ chai

Đây là vấn đề thật của một đội đồng người dùng chung một sản phẩm AI: ai trả tiền, ai được chạy, và chuyện gì xảy ra khi tài khoản đó chạm hạn mức vào 3 giờ chiều.

Hub trả lời bằng **hai lớp credential và một công tắc**.

- **Credential cá nhân.** Mỗi người tự upload `.credentials.json` của Claude CLI mình đang đăng nhập. Nó thuộc về người đó, không ai khác đọc được.
- **Credential dùng chung.** Admin publish một tài khoản Claude ở cấp workspace. Người mới vào đội chạy được ngay từ phút đầu, không cần có tài khoản Claude riêng.
- **Công tắc.** `PUT /credentials/claude/mode` đổi giữa `own` và `shared`. Quy tắc phân giải là **own → shared → none**: hub luôn ưu tiên credential của chính người đó, rơi về credential chung nếu không có, và nói thẳng "không có" thay vì im lặng thất bại.

Cái làm nó thành công cụ làm việc thật, chứ không phải một ô upload:

Token tự gia hạn được ghi ngược về hub. Access token của Claude OAuth sống vài giờ, nên một file `.credentials.json` thật gần như hết hạn ngay khi upload. CLI tự làm mới nó từ refresh token trong lúc chạy, và agent `PUT /credentials/claude/refreshed` để hub luôn là bản mới nhất. Hub theo dõi *có hay không có* refresh token — một boolean `has_refresh_token`, không bao giờ là chính token đó — nên một credential quá hạn nhưng gia hạn được hiện trạng thái `refreshable` chứ không bị bỏ đơ oan. Tín hiệu "cái này thật sự hỏng" chỉ đến từ một chỗ: CLI thật sự bị từ chối.

Chi tiêu hiện ra theo từng người. Mỗi lượt gọi Claude được `POST`

`/credentials/claude/usage` ghi lại token và cost, cộng dồn thành phần trăm hạn mức phiên và hạn mức tuần ngay trên chip credential ở header. Người dùng thấy còn bao nhiêu, chứ không đợi đến lúc bị chặn.

Bí mật được cất đúng cách. Credential mã hoá at-rest bằng `EMEHUB_ENCRYPTION_KEY`, tách hẳn khỏi `EMEHUB_JWT_SECRET` ký token — hai bí mật, không bao giờ suy ra từ nhau (ADR 0005). Thiếu bí mật thì hub **từ chối khởi động**, vì một khoá mã hoá tự sinh lúc boot sẽ lằng lể tạo ra những dòng dữ liệu không giải mã được sau lần restart kế tiếp.

Đúng một hàm trả về credential. `resolve_material` trong

`api/app/services/claude_credentials.py` là hàm duy nhất trong toàn bộ hub trả ra chất liệu credential. Mọi endpoint khác được khai báo với `response_model` không có trường nào chứa được nó, nên kể cả một lỗi lập trình trong handler cũng không serialise nổi token ra ngoài.

2. Spec được viết từ DOM thật, không phải từ trí tưởng tượng

Đây là điểm khác biệt lớn nhất về mặt kỹ thuật.

Cách thông thường để một LLM sinh test automation là: đọc mã nguồn, đoán selector, sinh file, chạy, hỏng, sửa. Selector đoán ra là nguồn gốc của gần như mọi spec chất yếu — nó trông hợp lý trong code review và không tồn tại trên trang thật.

Q-Agent có chế độ **live authoring**: một Claude tác tử **lái trình duyệt thật trước, viết spec sau**.

Cụ thể, qua CLI browser-harness gắn vào một Chrome đã đăng nhập sẵn:

1. **Thực hiện từng bước của test case trên app thật.** Không mô phỏng, không dry-run.
2. **Tìm phần tử qua accessibility tree, rồi xác minh.** Thứ tự ưu tiên selector là cố định và được ghi lại theo strategy: data-testid → ARIA role + tên → label → CSS ổn định. Không bao giờ :nth-child, không bao giờ class trần, không bao giờ selector dựa vào cấu trúc DOM.
3. **Xác minh selector, không phải toạ độ.** Đây là chỗ tinh tế nhất và cũng là chỗ dễ sai nhất. Click theo toạ độ trúng vào bất kỳ phần tử nào nằm ở điểm đó — thường là một thẻ <a> bên trong — trong khi spec sẽ click **selector đã ghi**, mà tâm của nó có thể là một container mà app không phản ứng. Nên mỗi thao tác được dispatch trực tiếp trên chính selector sắp đưa vào spec, rồi kiểm tra hiệu ứng thật (URL có đổi không). Một bước điều hướng chưa được nhìn thấy xảy ra qua selector đó thì không được tính là đã xác minh.
4. **Tự tạo dữ liệu test nếu chưa có.** Nếu case cần "một claim đang ở trạng thái nháp" mà hệ thống không có, agent tạo nó qua UI rồi ghi luôn bước tạo đó vào spec — để lần chạy sau spec vẫn tự đứng được, không phụ thuộc vào dữ liệu tình cờ tồn tại hôm nay.
5. **Chỉ khi đó mới emit spec**, dựng từ đúng những gì đã chạy được.

Kết quả là một file Playwright + TypeScript chạy xanh ngay, không cần một lượt heal nào.

Chế độ blind vẫn còn, và vẫn có ích. Với ứng dụng chưa chạy được hoặc case đơn giản, Q-Agent sinh spec từ Knowledge Base rồi self-heal: đưa lỗi kèm DOM sống ngược lại cho Claude, sửa có mục tiêu, chạy lại — tối đa 3 lượt, timeout rút ngắn, chạy trên model rẻ. Một lượt heal thành công có bám DOM sẽ ghi ngược selector **verified-at-runtime** vào Knowledge Base, nên lần sinh spec sau đã khôn hơn.

Và có một cửa chặn trước khi chạy. Spec sinh ra đi qua một *placeholder gate* — cửa kiểm tra trước khi thực thi — từ chối selector bịa, stub TODO và URL giữ chỗ **trước khi** spec được phép chạy; chất lượng spec còn có skill automation-reviewer soi riêng. Verdict của gate là passed / blocked / rejected. Một spec blocked hiện lên UI như "cần grounding thêm" — thường là dựng lại KB hoặc chạy một lượt exploration — thay vì lặng lẽ fail lúc execution và bị đọc nhầm thành lỗi sản phẩm.

Cả ba chế độ đều bị chặn bằng trần chi phí, trần số lượt và timeout theo đồng hồ.

3. Knowledge Base dựng từ mã nguồn thật

Trước khi sinh bất cứ thứ gì, project-bootstrap **đọc source thật của repo** — clone về nếu cần — để rút ra stack, kiến trúc, domain, route, selector thật, luồng đăng nhập, môi trường, và các Page Object / fixture có thể tái dùng. Kết quả lưu thành knowledge.md +

knowledge.json, **theo từng repository**, dựng một lần rồi mọi hành động AI phía sau đọc lại thay vì đọc lại code từ đầu.

Điều này đổi bản chất của prompt. requirement-analyst, test-case-generator và automation-generator không nhận một mô tả chung chung về "ứng dụng web"; chúng nhận base URL thật, route thật, selector thật và test account thật của chính project đó. Đó là lý do output dùng được gần như ngay, thay vì đầy chỗ trống cần điền tay.

Knowledge Base còn **giàu lên theo thời gian**: mỗi selector được xác minh trên app đang chạy được đóng dấu thời gian kèm strategy đã hoạt động, và mục verified-at-runtime luôn thẳng mục suy ra từ source, không bị lẫn merge sau ghi đè.

Một KB dựng đầy đủ tốn khoảng 20 phút, nên hub cho **clone** nó: admin dựng project mẫu trong namespace dùng chung, thành viên clone cả ProjectConfig, test account đã mã hoá và toàn bộ artefact trên đĩa về scope của mình — thay vì mỗi người chạy lại 20 phút đó.

4. Con người vẫn là người ký

Suite này cố tình không tự động hoá đến cùng. Có ba cửa mà một con người phải mở:

- **Review Center.** Test case do AI sinh ra vào trạng thái pending. Chỉ case được **approved** mới đi tiếp sang bước tạo trên provider và sang automation. Reviewer sửa được cả automation type của từng case (Playwright / Selenium / Cypress / Manual) — case Manual không bao giờ bị đem đi sinh spec.
- **Create & Link có chế độ dry-run.** Bước đẩy test case lên Azure DevOps có **local mode**: ghi lại phía mình, không viết gì lên provider. Không ai phải chọn giữa "thử sản phẩm" và "làm bản project thật của khách hàng".
- **Comment trước khi publish.** Kết quả trả về ticket là một comment được chuẩn bị và xem trước, cùng với mapping trạng thái cấu hình được, chứ không phải một lần ghi thẳng.

Mỗi lần fail còn được execution-analyzer phân loại: test_defect (spec sai) / product_defect (app sai) / flaky / environment / timeout. Việc này giữ cho một spec hỏng không bị đọc thành một bug sản phẩm — thứ làm hỏng niềm tin vào báo cáo tự động nhanh hơn bất cứ điều gì khác.

5. Bí mật nằm đúng chỗ nó phải nằm

Ba loại bí mật, ba nơi ở khác nhau, và không cái nào đi lang thang.

PAT của provider không rời khỏi hub. Hub giữ token Azure DevOps / Jira / GitHub đã mã hoá và **tự proxy** lời gọi provider. Agent không bao giờ cầm PAT. Endpoint trả về hasPat: true, không bao giờ trả về chính PAT.

Cookie đăng nhập của tester không rời khỏi máy tester. Ứng dụng nằm sau SSO/MFA cần một con người đăng nhập thật trong một trình duyệt có giao diện. **Local Agent** — một app Node chạy trên máy tester — thực thi spec ngay tại đó; cookie và storageState ở lại trên thiết bị, chỉ có spec, kết quả và evidence đi ngược về server. Máy đó tự chứng minh danh tính bằng **device pairing**: app phát một mã ghép đôi ngắn hạn, CLI đổi lấy token lâu dài của riêng thiết bị, server chỉ giữ bản hash, và admin thu hồi được bất cứ lúc nào.

Credential Claude đi ra ngoài đúng một lần, và được ghi nhận. Đây là ngoại lệ duy nhất và nó được viết thành văn bản, vì Claude CLI cần credential trên đĩa mới chạy được. Hub thu hẹp nó lại hết mức: agent trở CLAUDE_SECURESTORAGE_CONFIG_DIR — biến hợp, chỉ đi

dời đúng file credential — chứ không phải CLAUDE_CONFIG_DIR, vốn sẽ kéo theo cả skills/, settings.json và projects/.

Và một run dài hơn tuổi thọ của token. Access token của agent sống 15 phút; riêng bootstrap Claude của Q-Agent đã có timeout 1200 giây, một run đầy đủ còn dài hơn. Nên ADR 0009 đưa ra **run-scoped credential grant**: một giấy phép gắn với đúng một run, và nó chỉ với tới được **ba** route trong toàn bộ hub — lấy credential, ghi lại token đã gia hạn, ghi usage. Điều này không được kiểm tra bằng một câu if trong hàm xử lý grant, mà bằng chính cách wiring: không chỗ nào khác trong hub phụ thuộc vào require_credential_grant, và audience của grant không bao giờ đăng ký được, nên require_principal và require_user đều từ chối nó.

Không bao giờ fail-open. Không có cấu hình nào trong hub mà "xác thực không dùng được" dẫn tới "cho vào". Trang landing cũng theo quy tắc đó: đọc lỗi trạng thái sản phẩm nghĩa là **đóng**, chứ không phải mở.

Mọi việc đáng ghi đều vào **audit log** append-only: category, người thực hiện, hành động, đối tượng, IP, trạng thái, mã run.

6. Một lần đăng nhập, một địa chỉ, ba ứng dụng

Người dùng đăng nhập ở EmeHub rồi bấm Launch — không có màn đăng nhập thứ hai.

Cơ chế là hub mint một access token có audience gắn đúng một agent, ký bằng khoá của hub; agent tự xác thực token cục bộ, **không gọi ngược về hub theo từng request** (ADR 0008). Mỗi access token mang theo sid — id của phiên — nên **thu hồi một phiên là đăng xuất thiết bị đó khỏi mọi agent**, không phải ba lần thu hồi ở ba nơi.

Và cả suite nằm sau **một origin duy nhất**, agent gắn theo path (ADR 0010):

hub.chuongnd.click, hub.chuongnd.click/qagent/,

hub.chuongnd.click/dagent. Thanh địa chỉ không đổi khi chuyển ứng dụng. Nó đọc ra như một sản phẩm, vì nó là một sản phẩm.

7. Cấu hình một lần, ba ứng dụng cùng đọc

Hub là **nguồn sự thật** cho danh tính và cấu hình dùng chung (ADR 0001). Provider connection, project, repository, base URL, môi trường, test account, credential — khai báo một lần ở hub, agent đọc xuống.

Ranh giới được giữ chặt và có văn bản: **hub chỉ dựng những artefact mà nó đã sở hữu toàn bộ đầu vào** — hôm nay là knowledge base, và không gì khác (ADR 0007). Hub không sinh test, không sinh code, không lái trình duyệt, không tạo PR. Việc đó là việc của agent. Một tính năng thêm hành vi nghiệp vụ ngoài carve-out đó thì nó thuộc về agent, không thuộc về hub.

Hợp đồng giữa hub và agent là một tài liệu thật — INTEGRATION.md, 47 KB — và đối hợp đồng thì phải cập nhật nó trong cùng PR.

8. Chúng tôi tự khai những gì chưa làm

KNOWN-GAPS.md liệt kê thẳng: tính năng nào đã bị **gỡ khỏi UI** vì chưa thật, tính năng nào đang ẩn sau feature flag, chỗ nào còn nợ kỹ thuật, và cả những dòng trong tài liệu cũ mà chúng tôi kiểm chứng lại rồi phát hiện là **sai**.

Đây không phải một mục khiêm tốn hình thức. Nó là một quyết định kỹ thuật, ghi trong kế hoạch dọn dẹp: **một nút bấm trông như chạy được nhưng không làm gì thì phá uy tín nhiều hơn một tính năng không có mặt**. Trong đợt dọn trước khi nộp, chúng tôi đã gỡ đúng những nút như vậy — trong đó có một modal "Add knowledge" toast báo thành công mà không hề POST, và một trang Roles dựng trên bốn role tự nghĩ ra.

Một sản phẩm nói được điều nó chưa làm là một sản phẩm đáng tin ở những điều nó nói là đã làm.

Con số

Đếm được từ repo tại thời điểm viết:

Test backend của EmeHub	788 hàm test
Test backend của Q-Agent	1358 hàm test
Router API của hub	14
Màn hình UI của hub, chạy trên endpoint thật	11
Skill Claude chuyên biệt trong Q-Agent	14
ADR đã chốt trong hub	12
Ứng dụng trong suite	3 (EmeHub, Q-Agent, D-Agent)

Mỗi skill là một file SKILL.md riêng — phương pháp, luật chất lượng, template output — được backend nạp làm **system prompt** cho đúng hành động đó, trong khi prompt của lời gọi vẫn ghim chính xác hình dạng JSON mà backend sẽ parse. Không có prompt nào nằm rải rác trong code.

Đọc tiếp

- USER-GUIDE.md — dùng sản phẩm từ đăng nhập tới khi trả kết quả về ticket
- KNOWN-GAPS.md — những gì đã gỡ và những gì chưa làm
- INTEGRATION.md — hợp đồng giữa hub và agent
- ROADMAP.md — đang ở phase nào